

Secure Context Pipeline: PII Obfuscation for External LLM Providers

Design and build the system that lets users bring their most sensitive data into AI workflows — without ever exposing raw PII, PHI, or privileged content to an external model provider.

Time Allowed

8 – 10 Hours

Format

Take-Home / Async

Track

Security + AI · Senior

Confidentiality Notice: This challenge describes a real engineering problem YourAI is actively solving. Do not share this document, your solution, or any architectural details publicly or with third parties.

Business Context

YourAI is building an AI productivity platform for professionals who work with some of the most sensitive data that exists: medical records, legal case files, financial disclosures, and personal health information. Our users — physicians, attorneys, financial advisors, and compliance officers — need AI assistance with this data, but cannot expose it to third-party model providers in its raw form.

We use external LLM providers (OpenAI, Anthropic, and others) as inference backends. While we contractually require zero data retention on their end, contractual guarantees are not a sufficient security control. We need a **technical guarantee**: raw PII, PHI, or privileged content must never leave our infrastructure in a form that a provider — or any intermediary — can read or reconstruct.

At the same time, the obfuscated data fed to the LLM must retain enough semantic structure for the model to reason correctly and return a useful response. And when that response comes back, any references to obfuscated entities must be **transparently restored** to their original values before the user sees them.

The Problem You Are Solving

Data Type	Examples	Regulatory Scope
PII	Full name, SSN, DOB, address, email, phone	GDPR, CCPA, COPPA
PHI	Diagnoses, medications, MRN, insurance IDs, lab results	HIPAA (45 CFR Parts 160 & 164)
Legal Privilege	Case strategy, client identity, settlement terms, privileged communications	Attorney-Client Privilege, Work Product Doctrine
Financial PII	Account numbers, tax IDs, transaction history	GLBA, SOX, PCI-DSS

Your task is to design and implement the **Secure Context Pipeline** — the system layer that sits between YourAI's document store and the external LLM API. It must handle the full round-trip: detect → obfuscate → call LLM → restore.

System Architecture You Are Building



Core Requirements (Must Complete)

#	Component	Requirement
1	Secure Store	Users can upload documents (PDF, DOCX, TXT). Documents are encrypted at rest using AES-256. Each user's data is isolated.
2	PII/PHI Detector	Detect and classify entities across all four data types in the table above. Must identify at minimum: NAME, SSN, DOB, EMAIL.
3	Obfuscation Engine	Implement two strategies: (a) Tokenization — replace with typed tokens e.g. [PHI_NAME_a3f2] that are meaningless to the LLM but structured for restoration.
4	Session Vault	Store the bidirectional token ↔ original mapping, scoped to a user session. Vault entries must be encrypted. Same encryption as document store.

5	LLM Context Injector	Assemble the obfuscated document chunks into a context payload and call an external LLM API. The raw document c
6	De-obfuscation Engine	Parse the LLM response, detect all token references (including paraphrased or grammatically modified forms), and re
7	Audit Log	Every obfuscation and de-obfuscation event must be logged with: timestamp, session ID, entity type, token (not origin

Critical Constraint: The pipeline must be **reversible within a session** and **irreversible across sessions**. If a vault is destroyed, the original values cannot be recovered from tokens alone. This is a security guarantee, not a bug.

Non-Functional Requirements

- **Extensibility:** Adding a new entity type (e.g. PASSPORT_NUMBER) requires changes to the detector config only — zero changes to obfuscation or vault logic.
- **Strategy swappability:** Tokenization and pseudonymization are interchangeable via config — no business logic changes.
- **Zero PII in transit:** A network traffic inspection of the outbound LLM API call must show no recognizable PII, PHI, or privileged content.
- **Vault isolation:** A vault from Session A must be cryptographically inaccessible to Session B, even by the same user.
- **Graceful entity degradation:** If an entity type cannot be detected with sufficient confidence, it must be redacted entirely rather than passed through unmasked.
- **Async pipeline:** All I/O (storage reads, LLM calls, vault lookups) must be non-blocking.

Technology Stack

Category	Required / Accepted	Notes
Language	Python 3.10+	Async (asyncio) required throughout
PII Detection	spaCy, Microsoft Presidio, AWS Comprehend, or similar	Can be based on rule-based or ML-based, must justify choice — accuracy vs. latency vs. cost
Encryption	Python cryptography library (Fernet / AES-GCM)	No rolling your own crypto primitives
LLM Provider	OpenAI API or Anthropic API	Provided test key or candidate's own
Secure Storage	SQLite + SQLCipher, PostgreSQL, or encrypted filesystem	Must support per-user key isolation
Testing	pytest + pytest-asyncio	Required
Dev Tools	Git, Docker, .env config	Docker strongly encouraged for reproducibility
Cloud (optional)	AWS KMS / GCP KMS mock, S3 / GCS for doc store	Local mocks fully acceptable — design must be cloud-compatible

Expected Deliverables

Submit a private Git repository with the following structure:

```
/secure-context-pipeline
/store # Encrypted document upload + retrieval
/detection # PII / PHI / privilege entity detection
/obfuscation
/strategies # Tokenization + pseudonymization implementations
/engine # Orchestrates detection → strategy selection → output
/vault # Session-scoped encrypted token ↔ original mapping
/pipeline # End-to-end: ingest → obfuscate → LLM call → restore
/deobfuscation # Token detection + replacement in LLM responses
```

```
/audit # Compliance event logging  
/tests # Unit + integration tests  
docker-compose.yml # Reproducible local setup  
README.md # Architecture, threat model, tradeoffs, next steps
```

README must include: Threat model (what attacks does your design resist?), entity detection approach and why you chose it, a comparison of tokenization vs. pseudonymization for at least two entity types, known gaps, and what you would build next with an additional day.

Evaluation Rubric

Dimension	Wt.	What We Look For
Security Architecture	30%	Vault design is credible under adversarial review. Encryption is applied correctly (not just present). Zero PII/PHI leakage.
Obfuscation Quality	25%	Tokenization is deterministic within session, non-deterministic across sessions. Pseudonymization preserves semantics.
De-obfuscation Correctness	20%	All token references in LLM responses are restored, including grammatically inflected forms. No tokens leaked.
Code Quality	15%	Typed interfaces throughout. Strategies are pluggable without harness changes. Async I/O everywhere. Test coverage > 80%.
Critical Thinking	10%	README demonstrates awareness of inference attacks, vault as a high-value target, re-identification risk, and mitigation strategies.

Success Criteria

Functional Must-Haves

- ✓ A document containing PII/PHI is uploaded, stored encrypted, and retrievable only with the correct user key.
- ✓ All specified entity types are detected with labeled spans before any LLM call.
- ✓ The outbound LLM API payload contains zero instances of original PII/PHI values — verifiable by inspection.
- ✓ LLM response tokens are fully restored to original values in the output returned to the user.
- ✓ Session vault is destroyed on logout; subsequent sessions cannot access prior vault mappings.
- ✓ Audit log captures all obfuscation events with no original values present.
- ✓ End-to-end pipeline runs successfully on the provided test fixture document.

Performance Benchmarks

- Full pipeline (detect → obfuscate → LLM call → restore) for a 2,000-word document: **under 15 seconds**.
- Obfuscation engine alone (excluding LLM call) for a 2,000-word document: **under 2 seconds**.
- Vault lookup for token re-identification: **under 5ms per token**.
- De-obfuscation of a 500-token LLM response: **under 500ms**.
- Zero PII leakage verified across 100 automated test runs on varied fixture documents.

Code Quality Expectations

- All public interfaces fully type-annotated — entity detection, vault operations, obfuscation strategies.
- Obfuscation strategies implement a shared base class — adding a new strategy touches zero harness code.
- All secrets and keys via environment variables or .env — none hardcoded or committed.
- Tests cover: happy path, entity type not found, vault miss on de-obfuscation, expired session, and concurrent session isolation.
- Async I/O for all LLM API calls, vault reads/writes, and document store operations.
- **Logging must never emit original values** — token IDs only. This is a hard requirement.

Live Review Discussion (45 min, post-submission)

You will walk through your code with the team. Prepare to discuss:

- Q1.** Walk us through what happens when a user uploads a 50-page legal brief containing 200 distinct PII entities. Where are the bottlenecks and how would you address them at scale?
- Q2.** An LLM response contains the phrase 'the patient's condition.' Your token for the diagnosis was [PHI_DIAGNOSIS_x7a]. How do you handle co-reference — indirect references the model made without using the token?
- Q3.** Your pseudonymization engine replaces 'John Smith' with 'Michael Torres' consistently within a session. A sophisticated adversary has the obfuscated document and the LLM response. What can they infer, and how does your design limit that?
- Q4.** The session vault is now a single point of failure and a high-value attack target. Describe your threat model for it and what production-grade controls you would add.
- Q5.** A HIPAA auditor asks you to prove that no PHI was transmitted to the LLM provider in the past 30 days. What does your audit log give them and what gaps remain?
- Q6.** Your tokenization is deterministic within a session. Why is non-determinism across sessions a security requirement, not just a preference?

Submission Instructions

1. Push to a **private GitHub or GitLab repo** and grant access to the hiring team before the deadline.
2. README must include a working **setup guide** — reviewers should be running with `docker -compose up` or `pip install + python demo.py`.
3. Include a **demo script or notebook** that runs the full pipeline end-to-end on a provided or bundled test document.
4. Submit the repo link via the hiring portal **before the deadline**. Late submissions will not be reviewed.
5. **Do not open-source or publicly share** this challenge, the problem description, or your solution.